

✓ 03MIAR - Algoritmos de optimización

AG1- Actividad Guiada 1

William Rene Moreno

Colab: <https://colab.research.google.com/drive/1yRE2WBDlb8zbk8THzQOQxuHzJT84rLJg?usp=sharing>

github: https://github.com/renemorenov/VIU-AlgoritmosOptimizacion/blob/main/AG1/Algoritmos_WilliamReneMoreno_AG1.ipynb

Presentación

Profesor:

- Grupos A2, A3, B1, B2: David Álvarez Robles (dalvarezr@professor.universidadviu.com)

Objetivos

- Comprender la utilidad y aplicar algoritmos divide y vencerás
- Comprender la utilidad y aplicar algoritmos voraces
- Comprender la utilidad y aplicar algoritmos de backtracking
- Realizar ejercicios avanzados -> Búsqueda de distancias mínimas (opcional)

✓ Divide y vencerás - Problema Torres de Hanói

- Parto de un problema y lo soluciono mediante la solución de subproblemas y unión de resultados
- En este caso concreto: $H(n) = H(n-1) + 1 + H(n-1)$
- Pensar lo anterior como... "Muevo todos los discos salvo uno a torre auxiliar, muevo el disco más grande a la torre final y muevo todos los discos restantes de la torre auxiliar a la final"
- Solamente resuelto el problema de mover un disco (condición de parada de división) porque el resto se aplica el mismo algoritmo.
- Mover un disco es muy sencillo -> Solución directa -> Mover de torre de inicio a torre de fin
- Ojo, el print del "if" es mover un disco entre torres (cualesquiera) y el print del "else" es el caso de mover el disco grande a torre final

```
def torres_hanoi(numero_discos, inicio, fin, ejec=0):

    print(f"{'-'*ejec}torres_hanoi({numero_discos}, {inicio}, {fin})")

    # Si el número de discos es 1 -> Caso base, mover de inicio a fin
    if numero_discos == 1:
        #print(f"Caso base: Mover desde {inicio} a {fin} -- torres_hanoi({numero_discos}, {inicio}, {fin})")
        print(f"{'-'*ejec}Mover desde {inicio} a {fin}")
    # Si el número de discos es mayor que 1 -> Mover resto de discos, mover disco único,
    # recuperar => Recursivamente
    else:
        torres_hanoi(numero_discos=numero_discos-1, inicio=inicio, fin=6-inicio-fin, ejec=ejec+1)
        #print(f"Caso central: Mover desde {inicio} a {fin} -- torres_hanoi({numero_discos}, {inicio}, {fin})")
        print(f"{'-'*ejec}Mover desde {inicio} a {fin}")
        torres_hanoi(numero_discos=numero_discos-1, inicio=6-inicio-fin, fin=fin, ejec=ejec+1)

torres_hanoi(numero_discos=3, inicio=1, fin=3)
```

```
torres_hanoi(3, 1, 3)
--torres_hanoi(2, 1, 2)
--torres_hanoi(1, 1, 3)
--Mover desde 1 a 3
--Mover desde 1 a 2
--torres_hanoi(1, 3, 2)
--Mover desde 3 a 2
Mover desde 1 a 3
--torres_hanoi(2, 2, 3)
--torres_hanoi(1, 2, 1)
--Mover desde 2 a 1
--Mover desde 2 a 3
--torres_hanoi(1, 1, 3)
--Mover desde 1 a 3
```

Técnicas voraces - Problema del cambio

- Para resolver el problema, elijo una solución "buena" sin pensar en consecuencias
- Sería como elegir algo "óptimo" localmente con la esperanza de que sea lo mejor en general
- No siempre funciona bien -> Si en el conjunto de las soluciones, dos están próximas entre sí y generan una solución mayor puede fallar (ej: monedas [1, 3, 4, 9])

```
def cambio_monedas(sistema, cambio):

    print(f"Sistema (Monedas): {sistema}")
    print(f"Cambio: {cambio}")

    # Inicialización de resultados
    resultado = [0] * len(sistema)

    # Para cada posible moneda, calculamos el número de monedas máximo que podemos dar
    # y lo restamos del cambio pendiente (mejor decisión local, que no tiene por qué
    # ser óptima
    for i in range(len(sistema)):
        total_monedas = cambio//sistema[i] # División con resultado entero
        resultado[i] = total_monedas
        cambio = cambio - resultado[i]*sistema[i]

    return resultado

# Sistema óptimo
sistema = [25, 10, 5, 1]
cambio = 27
resultado = cambio_monedas(sistema,cambio)
print(f"Resultado: {resultado}")

print("")

# Sistema NO óptimo
sistema = [11, 5, 1]
cambio = 15
resultado = cambio_monedas(sistema,cambio)
print(f"Resultado: {resultado}")
```

```
Sistema (Monedas): [25, 10, 5, 1]
Cambio: 27
Resultado: [1, 0, 0, 2]
```

```
Sistema (Monedas): [11, 5, 1]
Cambio: 15
Resultado: [1, 0, 4]
```

Backtracking - Problema de las N reinas

- Se divide el problema en etapas
- En cada etapa se construye una solución parcial
- Se comprueba si esa solución parcial sigue siendo válida
- Si es válida, se continúa con la siguiente etapa
- Si no es válida, se descarta y se prueba otra alternativa
- Si ninguna alternativa funciona, se retrocede a la etapa anterior para cambiar decisiones

```
# Función que comprueba si por columnas hay un error en la solución
# etapa(i) = etapa (j) -> Error
# También comprueba si por diagonales hay un error en la solución
# (abs(i-j) == abs(xi-xj)) -> Error
def conflicto(solucion, etapa):
    for i in range(etapa+1):
        if solucion.count(solucion[i]) > 1:
            return True
    for j in range(etapa):
        if (i != j):
            if (abs(i-j) == abs(solucion[i]-solucion[j])):
                return True
    else:
        return False
```

```

# Búsqueda de una solución
def colocar_reinas(numero_reinas, solucion = None, etapa = 0):

    if solucion is None:
        solucion = numero_reinas*[0]

    # Caso base: si he colocado todas las reinas (he completado todas las etapas)
    # etapa comienza en 0 y llega hasta numero_reinas
    if numero_reinas == etapa:
        #print(f"({numero_reinas}, {solucion}, {etapa})")
        print(f"{solucion}")
        return solucion

    else:
        # Recorro todas las columnas posibles para la reina en la fila (etapa)
        for i in range(1,numero_reinas+1):
            solucion[etapa] = i
            #print(f"({numero_reinas}, {solucion}, {etapa})")

            # Compruebo si la colocación actual no entra en conflicto
            # con reinas anteriores (columnas y diagonales)
            if not (conflicto(solucion, etapa)):
                # Si es válida, intento resolver el resto del tablero
                resultado = colocar_reinas(numero_reinas, solucion, etapa + 1)

                # Si desde aquí se encuentra una solución válida,
                # la propago hacia arriba para detener la búsqueda
                # En otras palabras, ya tengo resultado, no necesito
                # seguir probando, lo devuelvo
                if resultado is not None:
                    return resultado

            # Limpio la posición de la reina si todos los estados fallan (vuelta atrás)
            solucion[etapa] = 0

        # Si ninguna columna funciona en esta etapa, esta rama no tiene solución
        #return None

solucion = colocar_reinas(4)

```

[2, 4, 1, 3]

```

# Búsqueda de todas las soluciones
def colocar_reinas(numero_reinas, solucion = None, etapa = 0):

    if solucion is None:
        solucion = numero_reinas*[0]

    # Caso base: si he colocado todas las reinas (he completado todas las etapas)
    # etapa comienza en 0 y llega hasta numero_reinas
    if numero_reinas == etapa:
        #print(f"({numero_reinas}, {solucion}, {etapa})")
        print(f"{solucion}")

    else:
        # Recorro todas las columnas posibles para la reina en la fila (etapa)
        for i in range(1,numero_reinas+1):
            solucion[etapa] = i
            #print(f"({numero_reinas}, {solucion}, {etapa})")

            # Compruebo si la colocación actual no entra en conflicto
            # con reinas anteriores (columnas y diagonales)
            if not (conflicto(solucion, etapa)):
                # Si es válida, intento resolver el resto del tablero
                resultado = colocar_reinas(numero_reinas, solucion, etapa + 1)

                # Si desde aquí se encuentra una solución válida,
                # la propago hacia arriba para detener la búsqueda
                # En otras palabras, ya tengo resultado, no necesito
                # seguir probando, lo devuelvo
                #if resultado is not None:
                #    return resultado

            # Limpio la posición de la reina si todos los estados fallan (vuelta atrás)
            solucion[etapa] = 0

```

```
# Si ninguna columna funciona en esta etapa, esta rama no tiene solución
#return None
```

```
soluciones = colocar_reinas(4)
```

```
[2, 4, 1, 3]
[3, 1, 4, 2]
```

```
def es_prometedora(solucion,etapa):
    for i in range(etapa+1):
        if solucion.count(solucion[i]) > 1:
            return False
        for j in range(i+1,etapa+1):
            if abs(i-j)==abs(solucion[i]-solucion[j]):
                return False
    return True
```

```
def reinas(N,solucion=[],etapa=0):
    if len(solucion) == 0:
        solucion = [0 for i in range(N)]
    for i in range(1,N+1):
        solucion[etapa] = i
        if es_prometedora(solucion,etapa):
            if etapa == N-1:
                print(solucion)
            else:
                reinas(N,solucion,etapa+1)
        else:
            None

    solucion[etapa] = 0
```

```
reinas(4)
```

```
[2, 4, 1, 3]
[3, 1, 4, 2]
```

```
# Búsqueda de todas las soluciones
soluciones = colocar_reinas(8)
```

```
[4, 2, 8, 5, 7, 1, 3, 6]
[4, 2, 8, 6, 1, 3, 5, 7]
[4, 6, 1, 5, 2, 8, 3, 7]
[4, 6, 8, 2, 7, 1, 3, 5]
[4, 6, 8, 3, 1, 7, 5, 2]
```

```
[6, 3, 7, 2, 8, 5, 1, 4]
```

```
[6, 3, 7, 4, 1, 8, 2, 5]
[6, 4, 1, 5, 8, 2, 7, 3]
[6, 4, 2, 8, 5, 7, 1, 3]
[6, 4, 7, 1, 3, 5, 2, 8]
[6, 4, 7, 1, 8, 2, 5, 3]
[6, 8, 2, 4, 1, 7, 5, 3]
[7, 1, 3, 8, 6, 4, 2, 5]
[7, 2, 4, 1, 8, 5, 3, 6]
[7, 2, 6, 3, 1, 4, 8, 5]
[7, 3, 1, 6, 8, 5, 2, 4]
[7, 3, 8, 2, 5, 1, 6, 4]
[7, 4, 2, 5, 8, 1, 3, 6]
[7, 4, 2, 8, 6, 1, 3, 5]
[7, 5, 3, 1, 6, 8, 2, 4]
[8, 2, 4, 1, 7, 5, 3, 6]
[8, 2, 5, 3, 1, 7, 4, 6]
[8, 3, 1, 6, 2, 5, 7, 4]
[8, 4, 1, 3, 6, 2, 7, 5]
```

▼ Actividades extra

Problema: Encontrar los dos puntos más cercanos

- Dado un conjunto de puntos se trata de encontrar los dos puntos más cercanos

Guía para aprendizaje:

- Suponer en 1D, o sea, una lista de números: [3403, 4537, 9089, 9746, 7259,]
- Primer intento: Fuerza bruta
- Calcular la complejidad. ¿Se puede mejorar?
- Segundo intento. Aplicar Divide y Vencerás
- Calcular la complejidad. ¿Se puede mejorar?
- Extender el algoritmo a 2D: [(1122, 6175), (135, 4076), (7296, 2741)....]
- Extender el algoritmo a 3D.

```
import math
num_list = [3403, 4537, 9089, 9746, 7259, 5462]
```

▼ Primer intento: Fuerza bruta:

```
def get_near_point(lista):
    distancia_minima = math.inf
    par_mas_cercano = None
    n = len(lista) #tamaño de la lista
    print("Tamño de la lista:",n)

    # Se compara cada elemento i con cada elemento j posterior:
    for i in range(n):
        for j in range(i+1, n):
            distancia = abs(lista[i] - lista[j])
            print(f"distancia entre {lista[i]} y {lista[j]}:", distancia)

            if distancia < distancia_minima:
                distancia_minima = distancia
                par_mas_cercano = (lista[i], lista[j])
                print(f"par_mas_cercano entre {lista[i]} y {lista[j]}:", par_mas_cercano)

    return par_mas_cercano, distancia_minima

resultado = get_near_point(num_list)
print(f"El par más cercano es {resultado[0]} con una distancia de {resultado[1]}")
print("Complejidad: O(n^2) => Complejidad Cuadrática")
```

```
Tamño de la lista: 6
distancia entre 3403 y 4537: 1134
par_mas_cercano entre 3403 y 4537: (3403, 4537)
distancia entre 3403 y 9089: 5686
distancia entre 3403 y 9746: 6343
distancia entre 3403 y 7259: 3856
distancia entre 3403 y 5462: 2059
distancia entre 4537 y 9089: 4552
distancia entre 4537 y 9746: 5209
distancia entre 4537 y 7259: 2722
```

```

distancia entre 4537 y 5462: 925
par_mas_cercano entre 4537 y 5462: (4537, 5462)
distancia entre 9089 y 9746: 657
par_mas_cercano entre 9089 y 9746: (9089, 9746)
distancia entre 9089 y 7259: 1830
distancia entre 9089 y 5462: 3627
distancia entre 9746 y 7259: 2487
distancia entre 9746 y 5462: 4284
distancia entre 7259 y 5462: 1797
El par más cercano es (9089, 9746) con una distancia de 657
Complejidad:  $O(n^2)$  => Complejidad Cuadrática

```

✓ Calcular la complejidad. ¿Se puede mejorar?:

```

"""
def get_near_point(lista):
    distancia_minima = math.inf #1
    par_mas_cercano = None #1
    n = len(lista) #1

    # Se compara cada elemento i con cada elemento j posterior:
    for i in range(n):
        for j in range(i+1, n): #n^2
            distancia = abs(lista[i] - lista[j]) #2 (1:resta + 1:abs)

            if distancia < distancia_minima: #1
                distancia_minima = distancia #1
                par_mas_cercano = (lista[i], lista[j]) #1

    return par_mas_cercano, distancia_minima #1
"""
print("Total operaciones:  $5(n^2) + 4$ ")
print("Complejidad:  $O(n^2)$  => Complejidad Cuadrática")

```

```

Total operaciones:  $5(n^2) + 4$ 
Complejidad:  $O(n^2)$  => Complejidad Cuadrática

```

✓ ¿Se puede mejorar?: Si

Al aplicar la fuerza bruta como en el caso anterior, se presenta mayor lentitud debido a que la comparación puede darse entre un numero con valor muy bajo y otro con un valor muy alto y así con todos los demas.

Se puede mejorar, ordenando la lista al iniciar la función, de esta manera se sabe con certeza que los números quedan lo mas cercanos posible entre si y con esto eliminamos la opción de tener un segundo bucle, ya que la comparación de cada elemento se efectua con su vecino inmediato.

De la siguiente forma quedaria con la mejora:

```

def get_near_point_optimized(lista):
    if len(lista) < 2:
        return None, 0

    # 1. Se ordena la lista
    lista.sort() # $O(n \log n)$  # Modifica la lista a: [3403, 4537, 5462, 7259, 9089, 9746]

    distancia_minima = math.inf
    par_mas_cercano = None

    # 2. Un solo bucle lineal para comparar numeros seguidos o adyacentes
    for i in range(len(lista) - 1): # n
        distancia = lista[i+1] - lista[i] #1 # Al estar la lista ordenada, el numero de la derecha siempre es mayor o igual que

        if distancia < distancia_minima: #1
            distancia_minima = distancia #1
            par_mas_cercano = (lista[i], lista[i+1]) #1

    return par_mas_cercano, distancia_minima

resultado = get_near_point_optimized(num_list)
print(f"El par más cercano es {resultado[0]} con una distancia de {resultado[1]}")
print("Total operaciones:  $O(n \log n) + O(n)$ ")
print("Complejidad:  $O(n \log n)$  => Complejidad - cuasi-lineal")

```

```
El par más cercano es (9089, 9746) con una distancia de 657
Total operaciones:  $O(n \log n) + O(n)$ 
Complejidad:  $O(n \log n) \Rightarrow$  Complejidad - cuasi-lineal
```

✓ Segundo intento. Aplicar Divide y Vencerás:

Para aplicar este algoritmo:

1. Ordenar lista
2. Dividir lista en 2
3. Calcular mejor distancia con fuerza bruta
4. Generar mínimos de cada uno de los 2 grupos

```
def get_near_point_dv(lista_ordenada):
    n = len(lista_ordenada)

    # Si la lista contiene maximo 3 elementos, se puede usar fuerza bruta directamente:
    if n <= 3:
        dist_min = math.inf
        par = None
        for i in range(n):
            for j in range(i+1, n):
                d = lista_ordenada[j] - lista_ordenada[i]
                if d < dist_min:
                    dist_min = d
                    par = (lista_ordenada[i], lista_ordenada[j])
        return par, dist_min

    # 2. Dividir lista en 2
    medio = n // 2
    izq = lista_ordenada[:medio]
    der = lista_ordenada[medio:]

    # 3. Calcular mejor distancia con fuerza bruta
    par_izq, dist_izq = get_near_point_dv(izq) #  $O(\log n)$ 
    par_der, dist_der = get_near_point_dv(der) #  $O(\log n)$ 

    # Encontrar el mínimo de los dos grupos
    if dist_izq < dist_der:
        delta = dist_izq
        mejor_par = par_izq
    else:
        delta = dist_der
        mejor_par = par_der

    # 3. COMBINAR (comparar el último del grupo izquierdo con el primero del derecho)
    dist_frontera = der[0] - izq[-1]
    if dist_frontera < delta:
        delta = dist_frontera
        mejor_par = (izq[-1], der[0])

    return mejor_par, delta

# 1. Ordenamiento de la lista:
num_list.sort() #  $O(n \log n)$ 
print("num_list: ", len(num_list))
mejor_par, delta = get_near_point_dv(num_list)

print(f"El par más cercano es {mejor_par} con una distancia de {delta}")
print("Complejidad:  $O(n \log n) \Rightarrow$  Complejidad - cuasi-lineal")
```

```
num_list: 6
El par más cercano es (9089, 9746) con una distancia de 657
Complejidad:  $O(n \log n) \Rightarrow$  Complejidad - cuasi-lineal
```

> ¿Se puede mejorar?

Si se puede lograr una mejora usando la lista original para enviar a la función cuando se usa recursivamente pasando los índices

Así queda con la mejora:

↳ 1 cell hidden

✓ Extender el algoritmo a 2D: [(1122, 6175), (135, 4076), (7296, 2741)...]

Para no caer en una complejidad como $O(n^2)$, se debe mantener la complejidad $O(n \log n)$ mediante un enfoque de Divide y Vencerás.

Pasos del algoritmo:

1. Crear 2 copias de cada par de números: la primera ordenada para el eje X y la otra para el eje Y.
2. Se divide trazando una línea vertical imaginaria en la mediana del eje X para dividir los puntos para en mitad Izquierda y mitad Derecha (Dividir).
3. De forma recursiva, se halla el par mas cercano de ambas mitades, quedandose con el delta de la distancia minima encontrada (Vencer).
4. Debido a que puede haber un par que cruza la línea de división; se construye una franja vertical centrada en la linea media, para analizar los pares de numeros dentro de esta franja; al ordenar los pares de números de dicha franja, matemáticamente se demuestra que para cada par solo necesita compararse con los siguientes en la lista ordenada.

Este tipo de algoritmos es frecuentemente usado al trabajar con sistemas de información geograficos, interactuando con coordenadas

```
import math

# Función auxiliar para calcular la distancia euclidiana entre dos puntos
def get_distancia_2d(p1, p2):
    print(f"Obtener distancia 2D entre {p1} y {p2}")
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

def get_par_mas_cercano_2d_rec(Px, Py):
    print("Obtener el par de numeros mas cercanos (en 2D)")
    n = len(Px)

    # Caso base: Si hay 3 o menos puntos, usamos fuerza bruta
    if n <= 3:
        dist_min = float('inf')
        par = None
        for i in range(n):
            for j in range(i + 1, n):
                d = get_distancia_2d(Px[i], Px[j])
                if d < dist_min:
                    dist_min = d
                    par = (Px[i], Px[j])
        return par, dist_min

    print("...1-Dividir")
    medio = n // 2
    punto_medio = Px[medio]

    # Dividir Px en dos mitades
    Px_izq = Px[:medio]
    Px_der = Px[medio:]

    # Dividir Py en dos mitades manteniendo el orden por Y
    # (Esto evita tener que ordenar Py en cada llamada recursiva, manteniendo el O(n))
    set_izq = set(Px_izq)
    Py_izq = [p for p in Py if p in set_izq]
    Py_der = [p for p in Py if p not in set_izq]
    print(f"...{Py_izq}; {Py_der}")

    print("...2-Vencer")
    par_izq, dist_izq = get_par_mas_cercano_2d_rec(Px_izq, Py_izq)
    par_der, dist_der = get_par_mas_cercano_2d_rec(Px_der, Py_der)

    # Seleccion del mínimo de los 2 lados:
    if dist_izq < dist_der:
        delta = dist_izq
        mejor_par = par_izq
    else:
        delta = dist_der
        mejor_par = par_der
    print(f"...delta:{delta}; mejor_par:{mejor_par}")

    print("...3-Combinar para Verificar numeros que cruzan la frontera")
    # Filtrar puntos dentro de la franja de ancho 2 * delta
    franja = [p for p in Py if abs(p[0] - punto_medio[0]) < delta]

    # Comparar cada punto de la franja con sus vecinos (máximo 7 comparaciones por punto)
```



```

tam_franja = len(franja)
for i in range(tam_franja):
    for j in range(i + 1, min(i + 8, tam_franja)): # El truco de los 7 vecinos
        d = get_distancia_2d(franja[i], franja[j])
        if d < delta:
            delta = d
            mejor_par = (franja[i], franja[j])

return mejor_par, delta

# Función Wrapper Principal
def buscar_par_mas_cercano_2d(puntos):
    print("Buscar el par de numeros mas cercanos (en 2D)")
    if len(puntos) < 2:
        return None, 0

    # Ordenamiento una sola vez por X y por Y
    Px = sorted(puntos, key=lambda p: p[0])
    Py = sorted(puntos, key=lambda p: p[1])

    return get_par_mas_cercano_2d_rec(Px, Py)

# --- EJEMPLO DE USO ---
lista_puntos = [(1122, 6175), (135, 4076), (7296, 2741), (1150, 6180), (5000, 5000)]
par, distancia = buscar_par_mas_cercano_2d(lista_puntos)

print(f"El par más cercano es: {par}")
print(f"Distancia euclidiana: {distancia:.2f}")

print("Complejidad:  $2T(n/2) + O(n) \Rightarrow O(n \log n) \Rightarrow$  Complejidad - Orden Logarítmico")

```

```

Buscar el par de numeros mas cercanos (en 2D)
Obtener el par de numeros mas cercanos (en 2D)
...1-Dividir
...[(135, 4076), (1122, 6175)]; [(7296, 2741), (5000, 5000), (1150, 6180)]
...2-Vencer
Obtener el par de numeros mas cercanos (en 2D)
Obtener distancia 2D entre (135, 4076) y (1122, 6175)
Obtener el par de numeros mas cercanos (en 2D)
Obtener distancia 2D entre (1150, 6180) y (5000, 5000)
Obtener distancia 2D entre (1150, 6180) y (7296, 2741)
Obtener distancia 2D entre (5000, 5000) y (7296, 2741)
...delta:2319.4762339804215; mejor_par:((135, 4076), (1122, 6175))
...3-Combinar para Verificar numeros que cruzan la frontera
Obtener distancia 2D entre (135, 4076) y (1122, 6175)
Obtener distancia 2D entre (135, 4076) y (1150, 6180)
Obtener distancia 2D entre (1122, 6175) y (1150, 6180)
El par más cercano es: ((1122, 6175), (1150, 6180))
Distancia euclidiana: 28.44
Complejidad:  $2T(n/2) + O(n) \Rightarrow O(n \log n) \Rightarrow$  Complejidad - Orden Logarítmico

```

✦ Extender el algoritmo a 3D

Para extender a 3D, se sigue la misma filosofía de **Divide y Vencerás**, pero cambiando las líneas por planos y los rectángulos por cubos:

1. Se traza un plano divisor bidimensional en la mediana del eje **X** (dividiendo el espacio tridimensional en 2: izquierdo y derecho).
2. La distancia euclidiana se actualiza en 3 dimensiones.
3. La franja se convierte en un especie de cuboide, luego de encontrar la distancia mínima en ambas mitades se revisa la frontera. De esta manera la franja se genera como un volumen tridimensional extendiéndose a una distancia delta a la izquierda y derecha del plano divisor.

```

import math

def get_distancia_3d(p1, p2):
    print("Calcular la distancia euclidiana en el espacio tridimensional.")
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2 + (p1[2] - p2[2])**2)

def get_par_mas_cercano_3d_rec(Px):
    print("Obtener el par de numeros mas cercanos (en 3D)")
    n = len(Px)

    if n <= 3: #Si hay 3 o menos puntos, se usa fuerza bruta
        dist_min = float('inf')

```

```

    par = None
    for i in range(n):
        for j in range(i + 1, n):
            d = get_distancia_3d(Px[i], Px[j])
            if d < dist_min:
                dist_min = d
                par = (Px[i], Px[j])
    return par, dist_min

print("...1-Dividir trazando el plano divisor en la mediana del eje X")
medio = n // 2
punto_medio = Px[medio]

Px_izq = Px[:medio]
Px_der = Px[medio:]

print("...2-Vencer usando recursividad para ambos lados del plano")
par_izq, dist_izq = get_par_mas_cercano_3d_rec(Px_izq)
par_der, dist_der = get_par_mas_cercano_3d_rec(Px_der)

# Nos quedamos con la menor distancia de los subproblemas
if dist_izq < dist_der:
    delta = dist_izq
    mejor_par = par_izq
else:
    delta = dist_der
    mejor_par = par_der

print("...3-Combinar para verificar numeros que cruzan la frontera")
# Filtramos los puntos que están dentro del "slab" tridimensional (ancho 2 * delta)
franja = [p for p in Px if abs(p[0] - punto_medio[0]) < delta]

# Ordenamos la franja por el eje Y para poder aplicar la ventana deslizante
franja.sort(key=lambda p: p[1])

tam_franja = len(franja)
for i in range(tam_franja):
    # Geométricamente en 3D, basta con comparar con los siguientes 15 vecinos
    for j in range(i + 1, min(i + 16, tam_franja)):
        d = get_distancia_3d(franja[i], franja[j])
        if d < delta:
            delta = d
            mejor_par = (franja[i], franja[j])

return mejor_par, delta

def buscar_par_mas_cercano_3d(puntos):
    print("Buscar el par de numeros mas cercanos (en 3D). Inicializa el ordenamiento en el eje X")
    if len(puntos) < 2:
        return None, 0

    # Ordenamos inicialmente por el eje X (esencial para la división del espacio)
    Px = sorted(puntos, key=lambda p: p[0])

    return get_par_mas_cercano_3d_rec(Px)

# Formato de los puntos: (X, Y, Z)
lista_puntos_3d = [
    (1122, 6175, 450),
    (135, 4076, 890),
    (7296, 2741, 120),
    (1150, 6180, 455), # Punto muy cercano al primero
    (5000, 5000, 5000)
]

par, distancia = buscar_par_mas_cercano_3d(lista_puntos_3d)

print(f"Punto 1: {par[0]}")
print(f"Punto 2: {par[1]}")
print(f"Distancia Euclidiana 3D: {distancia:.2f}")

print("Complejidad:  $2T(n/2) + O(n) \Rightarrow O(n \log n) \Rightarrow$  Complejidad - Orden Logarítmico")

```

Buscar el par de numeros mas cercanos (en 3D). Inicializa el ordenamiento en el eje X
 Obtener el par de numeros mas cercanos (en 3D)

...1-Dividir trazando el plano divisor en la mediana del eje X
...2-Vencer usando recursividad para ambos lados del plano
Obtener el par de numeros mas cercanos (en 3D)
Calcular la distancia euclidiana en el espacio tridimensional.
Obtener el par de numeros mas cercanos (en 3D)
Calcular la distancia euclidiana en el espacio tridimensional.
Calcular la distancia euclidiana en el espacio tridimensional.
Calcular la distancia euclidiana en el espacio tridimensional.
...3-Combinar para verificar numeros que cruzan la frontera
Calcular la distancia euclidiana en el espacio tridimensional.
Calcular la distancia euclidiana en el espacio tridimensional.
Calcular la distancia euclidiana en el espacio tridimensional.
Punto 1: (1122, 6175, 450)
Punto 2: (1150, 6180, 455)
Distancia Euclidiana 3D: 28.88
Complejidad: $2T(n/2) + O(n) \Rightarrow O(n \log n) \Rightarrow$ Complejidad - Orden Logarítmico